

CREATOR OS

MASTER BLUEPRINT v2.0

World-Class Creator Commerce Platform — Product, Architecture, Data, Security, Operations & Codex Build Plan

STATUS: LOCKED MASTER SOURCE OF TRUTH

This document consolidates the two prior Creator OS blueprints into one implementation source of truth. The original World-Class Blueprint supplies the product, data, security, finance, trust, analytics and roadmap requirements. The newer Architecture Decision Blueprint resolves the implementation stack for the greenfield build.

Technology supersession: the earlier blueprint listed Next.js + React + Node.js as its technology baseline. That is superseded by this v2.0 decision: **Laravel 13 + React 19 + TypeScript + Inertia 3**. The product and architectural requirements from the earlier blueprint remain in force unless explicitly changed in this master document.

Do not keep the previous two PDFs as active specifications after adopting this document. Use this master blueprint when instructing Codex and when making architecture decisions.

1. Executive Vision

Creator OS is a unified operating system for creator businesses: identity, storefront, commerce, subscriptions, content, messaging, bookings, live experiences, payments, payouts, CRM, analytics, trust and AI.

A creator should be able to publish an offer, acquire a fan, convert that fan into a paying customer, deliver value, receive money, understand the relationship and grow the business without stitching together many unrelated tools.

The strategic differentiation is business infrastructure rather than simply a prettier creator profile: reusable commerce primitives, granular entitlements, a real financial ledger, creator CRM, collaboration/revenue splits, fraud controls, analytics and AI assistance.

2. Final Technology Decision

Layer	LOCKED choice	Purpose
Backend/domain	Laravel 13 / PHP 8.3+	Business logic, transactions, auth, commerce, payments, ledger, jobs, notifications, admin, A
Web UI	React 19 + TypeScript	Mobile-first product experience.
Web application bridge	Inertia 3	Keep frontend/backend development cohesive without an unnecessary separate web API ap
UI	Tailwind CSS v4 + shadcn/ui	Foundation for a custom Creator OS design system.
OLTP database	PostgreSQL	Transactional truth for identity, commerce, payments, ledger and entitlements.
Cache/queues	Redis + Laravel Queues	Cache, rate limiting, short-lived state and background work.
Realtime	Laravel Reverb + Echo	WebSockets, presence, message delivery and realtime application events.
Calls	WebRTC + specialist media infrastructure	Audio/video transport and media sessions.
Live	Specialist ingest/transcoding/CDN	Scalable live video; never send video through normal Laravel HTTP requests.
Media	S3-compatible object storage + CDN	Private/public assets, variants and secure delivery.
Search	PostgreSQL first → OpenSearch when justified	Avoid premature infrastructure while preserving a clear extraction path.
Analytics	Transactional events → warehouse/OLAP when justified	Protect data integrity and correctness and enable creator/business analytics.
AI	Laravel AI SDK	Provider-flexible future copilot, tools, structured output, embeddings/RAG and agent workflow
Observability	OpenTelemetry-compatible approach + error monitoring	Tracing, metrics, logs, dashboards and alerting.

3. Architecture & Concurrency

The platform is a modular monolith first, with strong domain boundaries. Services are extracted only when scale, team ownership, deployment independence or reliability requirements justify it.

```
Clients
→ CDN / WAF
→ Load Balancer
→ Stateless Laravel 13 application nodes
→ PostgreSQL + Redis + Object Storage
→ Domain events / queues / notifications / analytics
→ Reverb cluster for realtime
→ WebRTC/media services for calls
→ Streaming ingest/transcoding/CDN for live
```

Concurrency is handled by separating workloads, not by choosing a particular language for everything. Laravel application nodes remain stateless and scale horizontally. Reverb owns persistent realtime connections. CDN/media infrastructure owns large media delivery. WebRTC/media services own audio/video. Redis and queues remove long-running work from request paths.

If a subsystem later benefits from Node.js, Go, Rust or another technology, extract only that subsystem behind a stable contract. Do not replace the core platform merely because one workload has different concurrency characteristics.

Reliability rule: if chat fails, video should not necessarily fail; if analytics is delayed, payment settlement must continue. Payment, entitlement and ledger paths receive the highest reliability priority.

4. Product Pillars

- **Identity:** verified creator identity, public profile, custom URL, social proof, roles and team access.
- **Commerce:** subscriptions, one-time products, PPV, bundles, tips, gifts, requests, tickets, courses, calls and events.
- **Content:** posts, images, video, audio, collections and access-controlled media.
- **Communication:** fan/creator messaging, notifications, realtime events, blocking and reporting.
- **Experiences:** bookings, voice/video calls, live streams and paid events.
- **Money:** payment intents, taxes/fees, double-entry ledger, refunds, disputes, holds and payouts.
- **Trust:** KYC, age checks, moderation, fraud/risk, abuse reporting, auditability and safety workflows.
- **Intelligence:** revenue analytics, funnels, cohorts, retention, creator CRM and AI copilot.

5. Personas & Roles

Fan/customer: discover creators, follow, buy, subscribe, message where permitted, book, attend and manage purchases.

Creator: publish, price, sell, communicate, schedule, view analytics and withdraw eligible earnings.

Creator team: role-based access for manager, editor, finance, moderator and support staff.

Platform operations: support, finance, trust & safety, moderation, risk, compliance and security.

Administrator: strictly controlled privileged access with just-in-time elevation and full audit trails.

6. Core User Journeys

Creator onboarding: Sign up → verify email/phone → choose creator handle → KYC/age verification → payout account → profile setup → create first product → publish.

Fan purchase: Visit profile → view offer → authenticate → payment intent → verified payment webhook → order recorded → entitlement granted → receipt/notification → content access.

Subscription lifecycle: Subscribe → active entitlement → renewal attempt → success/failure → grace period if applicable → cancellation/expiration → entitlement update.

Custom request: Fan creates request → creator accepts → payment authorized/collected → work delivered → fan confirmation or dispute window → funds become payable → payout.

Booking: Creator defines availability → fan selects slot → payment → booking lock → reminders → session token → completion/no-show/cancellation → settlement.

7. Generic Product & Entitlement Model

Every monetizable offer is a Product with type, price, currency, tax behavior, availability, inventory/capacity, delivery method, refund policy and entitlement rules.

Product types include Subscription, Digital Content, PPV, Bundle, Course, Ticket, Live Access, 1:1 Call, Group Call, Paid Request, Tip/Gift and future physical goods.

The Entitlement Engine is a first-class authorization layer. It answers what a user is entitled to access at a specific moment, preventing scattered access logic across features.

8. Domain Map

Domain	Responsibilities
Identity	Users, emails, phones, sessions, devices, MFA/passkeys, roles, permissions.
Creator	Profiles, verification status, teams, settings, storefront.
Commerce	Products, prices, variants, access rules, orders, subscriptions, bundles, PPV.
Payments	Payment intents, attempts, provider events, refunds, disputes, chargebacks, payout accounts, payouts.
Ledger	Accounts, transactions, entries, balance snapshots, fees, taxes, creator payable balances.
Entitlements	Access grants, expiration, revocation, subscription-derived access.
Content	Posts, media assets/variants, collections, access rules.
Messaging	Conversations, members, messages, attachments, blocks, reports.
Booking	Availability, exceptions, bookings, call sessions.
Live	Streams, viewers, tips, stream state and moderation hooks.
Trust & Safety	Risk events, moderation cases/actions, reports, audit logs.
Notifications	In-app, email/push abstractions and preferences.
Analytics	Canonical events, creator/admin reporting, warehouse pipeline.
AI	Provider abstraction, copilot, tools, RAG/knowledge base, AI audit/cost controls.
Admin/Ops	Support, finance, moderation, risk, security and operational actions.

9. PostgreSQL Database Blueprint

Identity: users, user_emails, user_phones, sessions, devices, roles, permissions, creator_profiles, creator_team_members, verification_cases.

Commerce: products, product_prices, product_variants, product_access_rules, orders, order_items, subscriptions, subscription_events, entitlements.

Payments: payment_intents, payment_attempts, payment_provider_events, refunds, disputes, chargebacks, payout_accounts, payouts, payout_items.

Ledger: ledger_accounts, ledger_transactions, ledger_entries, balance_snapshots.

Content: posts, media_assets, media_variants, collections, content_access_rules.

Communication: conversations, conversation_members, messages, message_attachments, blocks, reports.

Booking/live: availability_rules, availability_exceptions, bookings, call_sessions, live_streams, live_viewers, live_tips.

Trust/ops: risk_events, moderation_cases, moderation_actions, audit_logs, admin_notes, support_cases.

Analytics: event ingest metadata and warehouse-side fact/dimension models rather than overloading the OLTP database.

10. Financial Architecture

Non-negotiable: the browser never decides that money moved and never directly grants paid access.

Payment flow: Client creates payment intent → provider handles payment details → provider webhook is signature-verified → idempotent payment event is stored → order marked paid → ledger entries posted → entitlement granted.

Use immutable double-entry accounting. Every financial movement has balanced debit/credit entries and a traceable transaction identifier. Separate customer payment, platform fees, taxes, creator payable balance, refunds and payout movement.

Daily reconciliation compares external processor events with internal payment records and ledger balances. Exceptions enter an operations queue.

11. Authentication & Authorization

Use passkeys where available, secure OAuth, verified email/phone, optional or risk-triggered MFA, device/session management and suspicious-login detection.

Use RBAC for coarse roles plus resource/attribute checks for creator, team, product, entitlement and moderation access. Resolve requested objects and verify the actor's relationship and entitlement before returning data. Protect against IDOR/BOLA.

Privileged access requires separate admin identity controls, least privilege, short-lived elevation, strong MFA, risk controls and comprehensive audit logging.

12. Media & Content Security

Keep private media in non-public object storage. Deliver through short-lived signed URLs/tokens only after authorization.

For high-value media, support segmented streaming, expiring access, optional DRM and forensic watermarking where commercially justified. Track content identifiers and purchaser entitlements so leaks can be investigated and access revoked.

Uploads require virus/malware scanning, file-type validation, size limits, transcoding isolation and content moderation.

13. Messaging & Realtime

Messaging requires conversation-level authorization, rate limits, anti-spam, abuse reporting, attachment scanning, block/mute controls and retention rules.

Reverb/Echo provides realtime transport. The database remains authoritative. Presence, message delivery, booking state and live events may be realtime, but durable state must be persisted transactionally.

TLS is required in transit and encryption at rest. Do not promise end-to-end encryption unless the product can reconcile it with safety, support and lawful-access requirements.

14. Voice/Video & Live Streaming

Calls use WebRTC/media infrastructure with platform-issued session tokens. Never expose personal phone numbers. Session authorization must be tied to a valid booking or entitlement.

Live streaming is separated from realtime chat: specialist ingest/transcoding/CDN handles video, while Creator OS handles paid access, tickets, moderation, capacity, tips and safety.

Video/audio never passes through normal Laravel application requests.

15. Trust, Safety & Fraud

KYC uses a specialist regulated identity-verification provider. Store the minimum necessary result and provider reference rather than raw identity documents unnecessarily.

Risk signals include account age, device/IP reputation, transaction velocity, payment behavior, chargebacks, verification status, anomalous access and repeated abuse patterns.

Risk actions may include allow, step-up verification, hold, review, restrict, suspend or escalate according to policy.

Moderation combines automated detection, human review, creator/fan reports, evidence, case states and appeals.

16. Privacy & Data Governance

Classify data as public, internal, sensitive, highly sensitive, identity and financial, with different access and retention rules.

Collect only what is needed. Prefer processor references/tokens over payment credentials and verification results over raw identity documents. Define collection purpose, retention, deletion/anonymization, access requests, consent/notice where applicable and vendor data-processing responsibilities.

For an India-first launch, map applicable Indian privacy, tax, payments, intermediary/platform, consumer and content-safety obligations and obtain qualified legal review before launch.

17. Security Program

Secure SDLC: threat model → security review → code review → automated tests → dependency scanning → staging → penetration testing → production monitoring.

Controls include secrets management, key rotation, encryption, WAF, rate limits, API validation, secure headers, dependency pinning, least privilege, network segmentation and hardened CI/CD.

Testing includes unit, integration, contract, authorization, payment, webhook replay/idempotency, upload, abuse, load, chaos/failure and security regression tests. Plan independent penetration testing and progressively formalize controls toward SOC 2/ISO 27001-style governance if required.

18. Admin & Operations Console

Support: search users/orders/bookings, safe operational context, support cases and policy-bound actions.

Finance: refunds, disputes, chargebacks, payouts, holds, reconciliation and ledger investigation.

Trust & Safety: reports, moderation queues, risk alerts, restrictions and evidence.

Security: login anomalies, API abuse, privileged actions, audit events and incident response.

Admin tooling is production-critical software and must receive the same security discipline as the customer product.

19. Analytics & Data Platform

Emit canonical events such as `user.created`, `product.published`, `payment.completed`, `entitlement.granted`, `subscription.renewed`, `booking.completed` and `payout.completed`.

Pipeline: application events → durable event transport → warehouse/OLAP → curated metrics → dashboards/AI.

Core KPIs: GMV, net revenue, MRR, creator earnings, take rate, conversion, ARPU, LTV, churn, renewal rate, refund rate, chargeback rate, request completion and creator retention.

Do not send unnecessary sensitive payloads into analytics. Prefer stable internal IDs and aggregated/coarsened data where possible.

20. Creator CRM & AI

CRM segmentation may use engagement, purchase history, membership status, recency, frequency and monetary value, subject to privacy and policy.

AI copilot goals: explain revenue changes, identify funnel bottlenecks, draft offers, summarize performance, suggest content/product experiments and surface likely churn cohorts.

AI must not directly execute high-impact financial, moderation or account actions without explicit policy gates and human confirmation.

Laravel AI SDK is an integration foundation, not automatic intelligence. Provider keys, model choices, prompts, tools, safety controls, cost controls and evaluation still need to be deliberately designed.

21. Collaboration & Revenue Splits

Teams support managers, editors, finance and moderators through role-based access. Collaboration supports multiple creators on products/events with configurable revenue-share rules. Revenue split calculations feed the same immutable ledger rather than ad-hoc balance arithmetic.

22. API & Domain Design

Use versioned APIs where APIs are exposed, consistent error schemas, idempotency keys for financial mutations, cursor pagination, strict input validation and authorization at the domain boundary.

Representative domains: `/auth`, `/users`, `/creators`, `/products`, `/orders`, `/subscriptions`, `/entitlements`, `/payments`, `/payouts`, `/content`, `/messages`, `/bookings`, `/live`, `/reports`.

Webhooks require signature verification, timestamp/replay protection, idempotent processing, durable event storage and observability.

23. Mobile-First UX System

Design at 360/375/390/412px first, then expand to tablet and desktop. Use 44px+ touch targets, safe-area support, keyboard-aware forms, mobile bottom navigation, sheets/drawers for contextual actions, sticky primary actions where useful, responsive desktop navigation, accessible contrast/focus states and reduced-motion support.

Use strong typography, generous whitespace, subtle borders, restrained shadows and clear hierarchy. Avoid generic SaaS-template aesthetics, excessive gradients/glassmorphism, clutter, tiny text and gimmicky animation.

Core reusable UI primitives: Button, IconButton, Input, Textarea, Select, Checkbox, Radio, Switch, Avatar, Badge, Card, Sheet, Dialog, Drawer, Dropdown, Tabs, Toast, Alert, Skeleton, EmptyState, ErrorState and Confirmation.

24. Greenfield Repository Structure

```
app/  
  Domain/  
    Identity/ Creator/ Commerce/ Payments/ Ledger/ Entitlements/  
    Content/ Messaging/ Booking/ Live/ Trust/ Notifications/  
    Analytics/ AI/ Admin/  
    Actions/ Policies/ Jobs/ Events/ Listeners/ Http/ Support/  
resources/js/  
  components/ features/ layouts/ pages/ hooks/ lib/ types/  
tests/  
  Feature/ Unit/  
docs/  
  architecture.md security.md domain-model.md scaling.md decisions/
```

25. North-Star Architecture Rules

1. Money is never a mutable number without ledger evidence.
2. Access is never granted without an entitlement decision.
3. The browser is never trusted for payment success or authorization.
4. Private media is never permanently public.
5. Sensitive data is minimized and compartmentalized.
6. Every privileged action is attributable and auditable.
7. Analytics must not compromise transactional correctness.
8. Start modular; extract services when evidence demands it.

26. MVP Definition & Roadmap

MVP must-have: authentication; creator profiles; KYC workflow; products; memberships; one-time digital sales; payment integration; ledger; entitlements; media delivery; payouts; basic moderation; admin console; basic analytics.

Defer: full social discovery, advanced live, sophisticated AI, creator marketplace, complex collaboration, international tax/payment expansion and native apps until core economics and reliability are validated.

Phase 0 — Foundation: application foundation, threat model, data model, vendor evaluation, UX architecture, compliance mapping and engineering standards.

Phase 1 — Commerce Core: identity, creators, products, checkout, ledger, entitlements, content and payouts.

Phase 2 — Relationship Layer: messaging, CRM, bookings, requests, notifications and richer analytics.

Phase 3 — Experiences: live, calls, events, bundles, collaboration and revenue splits.

Phase 4 — Intelligence: AI copilot, churn prediction, recommendations, advanced fraud/risk and automated creator growth tools.

Phase 5 — Global: multi-currency, multi-language, regional payments/tax/compliance and global infrastructure.

27. Correct Build Order

1. Freeze product vocabulary and domain boundaries.
2. Create ERD and migration-ready PostgreSQL schema.
3. Define API contracts and authorization matrix.
4. Design payment state machine and double-entry ledger.
5. Design entitlement/access-control engine.
6. Design media pipeline and private delivery.
7. Build creator/fan/admin UX around those primitives.
8. Add observability, fraud, moderation and analytics.
9. Security test and load test.
10. Expand into live, calls, AI and global features only after the core is reliable.

28. Phase 0 — Exact Codex Mission

Codex must build the foundation directly in the intentionally fresh workspace. Do not create a nested application. Inspect the current Laravel 13 official React starter requirements and use current official package/docs versions rather than guessing commands.

Phase 0 must establish Laravel 13 + React 19 + TypeScript + Inertia 3 + Tailwind 4 + shadcn/ui, PostgreSQL, Redis, official authentication, mobile-first application shell, design tokens, domain folders, authorization conventions, queue/event/notification foundations, Reverb readiness, AI SDK foundation if cleanly supported by the current official package/docs, tests, formatting, lint/type checks, production build checks and architecture/security documentation.

Do not implement payments, KYC, video calls, live streaming or production AI features during Phase 0. The goal is a clean, production-minded foundation—not fake feature completeness.

29. MASTER CODEX PROMPT

Paste the following as the first instruction to Codex from the fresh project directory.

You are the lead staff engineer building CREATOR OS, a greenfield, world-class creator commerce and monetization platform.

WORKSPACE:

The current directory is intentionally fresh. Build directly in the current workspace. Do not create a nested application. Inspect the workspace before executing commands.

LOCKED STACK:

- Laravel 13
- PHP 8.3+
- React 19
- TypeScript
- Inertia 3
- Tailwind CSS v4
- shadcn/ui customized into the Creator OS design system
- PostgreSQL
- Redis
- Laravel Queues
- Laravel Reverb + Echo
- Object Storage + CDN abstraction
- WebRTC + specialist media infrastructure for future voice/video
- Specialist streaming infrastructure + CDN for future live streaming
- Laravel AI SDK for future AI capabilities

ARCHITECTURE:

Laravel is the business/domain brain.

React is the web UI.

Inertia connects them.

Reverb handles realtime application events/connections.

WebRTC/media infrastructure handles audio/video.

Streaming infrastructure handles ingest/transcoding/delivery.

CDN handles large media delivery.

PostgreSQL is transactional truth.

Redis handles cache/queues/rate limits/shared short-lived state.

Use a modular monolith first. Extract services only when evidence justifies it.

IMPORTANT:

Do not switch to Next.js/Node as the primary application stack.

Do not use Blade as the primary UI architecture.

Do not build video transport through Laravel.

Do not prematurely create microservices.

Do not implement payments, KYC, payouts, video calls, live streaming or production AI features in Phase 0.

FIRST:

Inspect the current workspace.

Inspect current official Laravel 13 React starter-kit requirements and current official package documentation.

Use current official package versions and commands rather than guessing.

If a partial Laravel installation exists, reuse it cleanly rather than creating a nested project.

Do not perform destructive operations without explaining them.

PHASE 0:

1. Establish official Laravel 13 React starter architecture.
2. Establish React 19 + TypeScript + Inertia 3.
3. Establish Tailwind CSS v4 + shadcn/ui foundation.
4. Establish PostgreSQL and Redis configuration.
5. Establish official authentication foundation.
6. Establish mobile-first Creator OS application shell.
7. Establish Creator OS design tokens.
8. Establish domain-oriented backend structure.
9. Establish frontend feature structure.
10. Establish authorization/policy conventions.
11. Establish queue/event/listener/job/notification conventions.
12. Establish Reverb readiness.
13. Establish AI SDK foundation only if cleanly supported by current official docs/package; no production AI feature.
14. Establish tests and quality gates.
15. Establish architecture/security/scaling documentation.

BACKEND DOMAINS:

app/Domain/Identity
app/Domain/Creator
app/Domain/Commerce
app/Domain/Payments
app/Domain/Ledger
app/Domain/Entitlements
app/Domain/Content
app/Domain/Messaging
app/Domain/Booking
app/Domain/Live
app/Domain/Trust
app/Domain/Notifications
app/Domain/Analytics
app/Domain/AI
app/Domain/Admin

Also establish appropriate Actions, Policies, Jobs, Events, Listeners, Http and Support conventions.

FRONTEND:

resources/js/
components/
features/
layouts/
pages/
hooks/
lib/
types/

MOBILE-FIRST DESIGN:

Start at 360/375/390/412px.
Use 44px+ touch targets.
Support safe areas and keyboard-aware forms.
Use mobile bottom navigation and responsive desktop navigation.
Use sheets/drawers for contextual actions.
Provide loading, empty and error states.
Use accessible focus/contrast behavior and reduced-motion support.
Avoid generic SaaS-template aesthetics.

DESIGN SYSTEM:

Create tokens for colors, typography, spacing, radii, borders, shadows, motion, breakpoints, z-index and safe-area behavior.
Create reusable primitives including Button, IconButton, Input, Textarea, Select, Checkbox, Radio, Switch, Avatar, Badge, Card, Sheet, Dialog, Drawer, Dropdown, Tabs, Toast, Alert, Skeleton, EmptyState, ErrorState and Confirmation.

SECURITY:

- Never trust browser authorization, price, payment status or entitlement.
- Server-side authorization for every protected resource.
- Protect against IDOR/BOLA.
- Secure sessions/cookies and CSRF.
- Sensible rate limiting.
- No secrets in source or logs.
- Security-conscious logging and audit conventions.
- Prepare signed private media URLs.
- Prepare webhook idempotency/signature verification patterns.
- Never store raw payment-card data.
- Design future KYC as provider-based and data-minimized.

DOMAIN PRINCIPLES:

Commerce is product-based.
Entitlements determine access.
Financial architecture uses an immutable double-entry ledger.
Important operations are idempotent.
Domain events support asynchronous processing.
Long-running work belongs in queues.
Realtime events are separate from transactional state.
Analytics must not compromise transactional correctness.

SCALING:

Application nodes must be stateless and horizontally scalable.
Redis is shared infrastructure where appropriate.
Reverb is the realtime layer.

CDN/media infrastructure handles large media delivery.
Video/audio never passes through normal Laravel HTTP requests.
Avoid N+1 queries.
Use pagination/cursors.
Keep transactions short and explicit.
Do not introduce microservices without a documented reason.

DOCUMENTATION:

Create:
docs/architecture.md
docs/security.md
docs/domain-model.md
docs/scaling.md
docs/decisions/
README.md

QUALITY:

Run and verify backend tests, authorization tests, frontend type checking, linting, formatting, production build and relevant accessibility checks.
Add CI if supported by the starter architecture.

FINAL REPORT:

When complete, report exact files/folders created, commands executed, packages installed and why, database/configuration assumptions, security measures, tests and results, build result, remaining gaps and recommended Phase 1 plan.

Actually implement the foundation. Do not merely describe it.

30. Launch Readiness Checklist

Security: threat model complete; secrets managed; authorization tests; dependency scanning; penetration test; incident response plan.

Payments: webhook signatures; idempotency; reconciliation; refund/dispute flows; ledger balancing; payout controls.

Privacy: data map; retention schedule; deletion/access workflows; vendor review; privacy notices and legal sign-off.

Operations: support tooling; moderation queue; fraud review; finance reconciliation; backup restore test; monitoring and alerts.

Performance: load test checkout, content access, messaging and creator dashboards; define SLOs and capacity thresholds.

31. Final Recommendation

LOCKED FINAL DECISION: Creator OS will be built as a Laravel 13 + React 19 + TypeScript + Inertia 3 modular monolith, backed by PostgreSQL and Redis, with Reverb for realtime and specialist infrastructure for WebRTC/live media. The original World-Class Blueprint's product, financial, security, trust, analytics and operations requirements remain part of the system's target architecture.

The strongest version of this company is not a clone of an existing creator-pass product. It is a secure, extensible Creator Operating System whose primitives—identity, products, payments, ledger, entitlements, content, relationships, trust and data—support many monetization models without redesigning the platform each time.

Source consolidation: the prior World-Class Blueprint contained the broad product/technical requirements; the Architecture Decision Blueprint resolved the greenfield implementation stack and Phase 0. This master document combines both and explicitly records the technology decision that supersedes the earlier Next.js/Node baseline.